

COMIC CRAFT AI

AI-Powered Comic Creation Platform

Project Detail	Information
Project Name	COMIC CRAFT AI
Team ID	SWTID-2026-6682
Date	30 September 2026
Project Type	Generative AI web application

Project Documentation

Prepared to describe the project purpose, system architecture, workflow, development activities, testing approach, deployment considerations, and future enhancements.

1. Project Description

Comic Craft AI is an AI-assisted web application intended to help users turn a story idea or prompt into a comic concept. The user describes the scene, characters, theme, or story direction; the application sends the request to a generative AI service and presents a structured response that can support comic planning and creation.

The project uses a Python web backend with a browser-based interface. Based on the project setup, the backend is built with FastAPI, page rendering uses Jinja2 templates, and AI integration is handled through a Gemini-related module. The application can be extended over time with panel-by-panel story planning, character consistency tools, image generation, and export options.

Project Objectives

- Provide a simple interface for entering a comic idea and describing the desired story or scene.
- Use generative AI to assist with story development, dialogue, scene descriptions, and comic-panel planning.
- Present the generated content clearly so users can review and refine it.
- Keep the backend modular so additional comic creation features can be added without redesigning the whole application.
- Handle user input and AI-service errors gracefully and protect API credentials.

Scope

The core scope is AI-assisted comic ideation and content generation through a web interface. Features such as image generation, saved projects, user accounts, collaborative editing, and downloadable comic books are suitable extensions unless they are already implemented in the running version.

Example Usage Scenarios

Scenario	User Request	Expected Application Support
1. Fantasy adventure	A young inventor discovers a hidden city in the clouds.	Develop a short story premise, character descriptions, scene progression, and panel ideas.
2. Educational comic	Explain recycling to school students through friendly characters.	Suggest a simple sequence of panels, accessible dialogue, and a clear learning message.
3. Superhero scene	A hero protects a town during a sudden storm.	Create an action sequence with visual descriptions, dialogue, and a beginning, middle, and ending.

2. Technical Architecture

The application follows a modular web architecture. The browser presents the input form and generated results. FastAPI receives requests, validates input, calls the AI integration module, and returns the result for display. The Gemini service processes the prompt and returns generated text. The precise deployed components should be confirmed against the current source code and environment.

Layer	Responsibility	Technology / Component
Presentation	Collect story prompts and show generated comic content.	HTML, CSS, Jinja2 templates, browser
Application backend	Routes, validation, prompt preparation, response handling.	Python, FastAPI
AI integration	Send prompts to the configured generative model and handle responses.	Gemini integration module / API

Layer	Responsibility	Technology / Component
Configuration	Load secrets and environment-specific settings.	Environment variables and local configuration
Optional persistence	Save projects, user preferences, or generated assets if added.	Database / file or object storage, if implemented

Request Flow

- The user opens the Comic Craft AI home page in a browser.
- The user enters a story idea and any supported options, then submits the form.
- The FastAPI route validates the request and prepares a prompt for the AI service.
- The AI integration module sends the prompt to the configured Gemini service.
- The backend handles the response or an error and passes the result to the page.
- The interface displays the generated content so the user can review or refine it.

Architecture Notes

Keep API keys outside source code, avoid logging secrets, set sensible request limits, and return clear error messages when the AI service is unavailable. If the application later stores user projects or generated images, add a documented data model and an ER diagram for the database.

3. Prerequisites and Development Setup

The following skills and tools are relevant to building and maintaining the application:

- Python programming and virtual-environment management.
- FastAPI routing and request/response handling.
- HTML, CSS, JavaScript, and Jinja2 template rendering.
- Generative AI API integration, prompt design, and response parsing.
- Git for version control and dependency management.
- Environment variables for API credentials and configuration.

Suggested Setup Sequence

Step	Activity	Purpose
1	Install a supported Python version.	Provide the runtime for the backend.
2	Create and activate a virtual environment.	Isolate project dependencies.
3	Install dependencies listed in the project's requirements file.	Install FastAPI, server, template, and AI client libraries as configured.
4	Create a local environment configuration file.	Store the AI API key and settings outside source code.
5	Start the development server using the project's configured command.	Run the application locally for verification.
6	Open the local URL in a browser and test the main workflow.	Confirm that the form, backend, and AI integration work together.

Use the exact package names, environment-variable names, and launch command already defined by the project. Do not commit secret configuration files to version control.

4. Project Workflow and Milestones

Milestone	Activities	Deliverable
1. Requirements and architecture	Define the target users, expected comic workflow, input fields, AI response format, and component boundaries.	Requirements and architecture overview
2. AI integration	Configure credentials securely, prepare prompts, call the AI service, and handle response failures.	Working AI request/response path
3. Backend implementation	Implement FastAPI routes, validate requests, invoke the AI module, and return useful results.	Backend endpoints
4. Frontend development	Create the prompt form, layout, loading state, result display, and error messages.	Usable browser interface
5. Testing and refinement	Test normal inputs, empty prompts, long prompts, service errors, and display behavior.	Test results and fixes
6. Deployment and documentation	Configure the runtime, secrets, logging, deployment settings, and user instructions.	Deployable application and project report

5. Milestone 1 — Requirements and Architecture

Activity 1.1: Define User Requirements

Identify the kind of comic users want to create and the minimum information needed to guide generation. Useful prompt details may include genre, target audience, characters, setting, mood, story length, and visual style. Only include options supported by the current interface.

Activity 1.2: Define the AI Output

Choose a consistent response structure so generated content is easy to render. A useful comic-planning response may contain a title, a short synopsis, character notes, and numbered panels with scene descriptions and dialogue. Validate the response before displaying it.

Activity 1.3: Design the Application

- Keep routes, templates, and AI service logic separated where practical.
- Use a clear interface between the FastAPI route and the Gemini integration module.
- Handle missing inputs and AI service errors without exposing internal stack traces to users.
- Document assumptions about generated output and any limitations of the selected model.

6. Milestone 2 — AI-Assisted Comic Generation

Prompt Preparation

Construct prompts from the user's story idea and supported settings. Give the model clear instructions about the intended audience, story structure, panel count when applicable, dialogue style, and the format of the response. Ask the model to keep characters and events consistent throughout the comic.

Response Handling

Treat AI output as generated content that may need validation. Check that expected fields are present, handle empty or malformed responses, and provide a recoverable error state if the external service fails. Avoid assuming every AI response will follow the requested format perfectly.

Potential Output Sections

Output Section	Purpose
Title and premise	Summarize the comic concept in a few lines.
Characters	Describe each character's role and notable traits.
Panel sequence	Arrange scenes in a logical visual order.
Dialogue and captions	Provide short speech or narration appropriate to each panel.
Visual direction	Describe setting, action, composition, and mood for artists or image-generation tools.

7. Milestone 3 — Backend Development

FastAPI Responsibilities

- Serve the home page and connect it to the correct Jinja2 template.
- Receive user submissions through the configured route or form handler.
- Validate required fields, data types, and reasonable length limits.
- Call the AI integration module and normalize the response.
- Return generated content to the template or frontend in the expected format.
- Log operational errors safely without revealing API keys or sensitive data.

Error Handling

Condition	Expected Behavior
Empty story prompt	Ask the user to enter a story idea before generation.
Input exceeds configured limit	Show a clear message and request a shorter prompt.
AI API key missing or invalid	Show a configuration/service error and log diagnostic details safely.
AI service timeout or rate limit	Display a retry-friendly message and avoid a broken page.
Unexpected response format	Handle the response gracefully and do not render malformed data as if it were valid.

Implementation details such as route names, exact request schemas, model identifiers, and response fields should match the current Comic Craft AI codebase.

8. Milestone 4 — Frontend Development

Interface Structure

The interface should make the comic-creation flow easy to understand: a project heading, a prompt-entry area, optional supported settings, a clear generate action, and a result section. Use consistent spacing, readable typography, and responsive layout behavior.

Usability Requirements

- Show a loading indicator while generation is in progress.
- Prevent accidental repeated submissions while a request is running.
- Display the result in readable sections, with panel numbers and dialogue clearly distinguished.
- Show helpful messages for validation errors and service failures.

- Ensure the layout works on desktop and mobile screens.
- Use accessible labels, keyboard navigation, and adequate color contrast.

Template Integration

Jinja2 templates can render the initial page and any server-provided result data. If JavaScript is used for asynchronous generation, keep the request format and response structure aligned with the FastAPI handler. Escape or safely render generated text to reduce the risk of HTML/script injection.

9. Milestone 5 — Testing and Deployment

Testing Checklist

Test	Expected Result
Valid comic prompt	The application returns and displays a useful comic concept.
Empty prompt	The user receives a clear validation message.
Long prompt	The configured input limit is enforced.
AI service unavailable	A readable error is shown and the page remains usable.
Malformed AI response	The backend handles the response safely.
Repeated submissions	The interface avoids confusing duplicate requests where possible.
Mobile browser	Form controls and generated content remain readable.
Secret handling	API keys are not exposed in source control, HTML, or routine logs.

Local Run and Verification

Start the application with the command defined by the project's README or environment configuration. Open the local address printed by the server, submit a test prompt, and verify the complete flow from browser input through FastAPI and the AI integration to the displayed result.

Deployment Considerations

- Use a production-appropriate ASGI server configuration for deployment.
- Store API keys and other secrets in environment configuration or a managed secret store.
- Disable development debug behavior in production.
- Use HTTPS, access controls where appropriate, request limits, and monitoring.
- Check AI provider usage limits, latency, and costs before sharing the application widely.
- If using a tunnel for a demonstration, treat it as temporary access rather than a substitute for production hosting.

10. Security, Privacy, and Reliability

Comic prompts may contain unpublished story ideas or personal information. Collect only what is needed, explain how data is handled, and avoid storing prompts unless project persistence is intentionally implemented.

- Keep API credentials in environment variables and exclude local secret files from Git.
- Validate and limit user input on the server, even if the browser also validates it.
- Escape user-provided and AI-generated text before rendering it in HTML.
- Use timeouts and handle provider errors and rate limits.

- Avoid logging full prompts or responses when they may contain sensitive material.
- Add authentication, authorization, backups, and retention policies if user accounts or saved projects are introduced.

11. Scalability and Future Enhancements

Area	Possible Enhancement	Expected Benefit
User load	Run multiple application instances behind a load balancer when demand requires it.	Support more concurrent users.
Generation time	Move long-running AI tasks to a background queue.	Keep the interface responsive.
Storage	Use managed database and object storage if saved projects or image assets are added.	Organize and retrieve larger project collections.
Character consistency	Maintain reusable character profiles and style guidance.	Improve continuity across panels.
Image generation	Integrate an image-generation service if within project scope.	Turn panel descriptions into visual drafts.
Export	Offer a downloadable storyboard or comic layout.	Make generated work easier to share and review.
Collaboration	Add accounts, saved projects, and team editing with access controls.	Support longer projects and team workflows.
Language support	Add multilingual prompts and dialogue options.	Make the tool useful to a wider audience.

Suggested Roadmap

Phase	Planned Enhancement	Indicative Timeline
Phase 1	Stabilize prompt submission, AI response handling, and result presentation.	Current / initial
Phase 2	Improve panel structure, validation, error handling, and usability.	1–2 months
Phase 3	Evaluate saved projects, background jobs, and image-generation integration.	3–4 months
Phase 4	Explore collaboration, export options, stronger monitoring, and expanded language support.	5–6 months

Roadmap timelines are planning estimates and should be adjusted to match team capacity and actual implementation progress.

12. Conclusion

Comic Craft AI applies generative AI to support comic planning and story creation through a web application. The FastAPI backend, Jinja2-based page rendering, and Gemini integration provide the foundation for accepting a story idea, requesting AI-generated content, and presenting the response to the user.

The project can grow through improved panel structuring, character consistency, image generation, export features, and saved projects. Continued attention to validation, safe credential handling, usability, and reliable error handling will help make the application more maintainable and useful.

Project Information

Field	Value
Project Name	COMIC CRAFT AI
Team ID	SWTID-2026-6682
Date	30 September 2026
Team Members	S Shanmugapriya (Team Leader); G vidhya; Karthigeyan; O Sukran